

En este laboratorio, implementaremos una arquitectura robusta para gestionar datos híbridos. El objetivo es que la aplicación sea capaz de sincronizar información desde una nube externa (**ReqRes.in**) y mantener una copia funcional en una base de datos local (**Isar**), permitiendo el acceso a la información incluso sin conexión.



Estructura de Directorios del Proyecto

Para mantener un código escalable y organizado, utilizaremos una estructura basada en capas. Asegúrate de crear los siguientes **9 archivos**:

```
lib/
├── config/
│   ├── dio_config.dart
│   └── isar_config.dart
├── domain/
│   └── models/
│       └── app_user.dart
├── infrastructure/
│   ├── datasources/
│   │   ├── isar_datasource.dart
│   │   └── remote_user_datasource.dart
│   └── repositories/
│       └── user_repository_impl.dart
├── presentation/
│   ├── providers/
│   │   └── user_provider.dart
│   └── screens/
│       └── user_list_screen.dart
└── main.dart
```

0. Dependencias

```
dependencies:
  flutter:
    sdk: flutter

  # Comunicación HTTP
  dio: ^5.4.1

  # Persistencia Local (Base de Datos NoSQL)
  isar: ^3.1.0+1
  isar_flutter_libs: ^3.1.0+1
  path_provider: ^2.1.2

  # Gestión de Estado Proactiva
  flutter_riverpod: ^2.4.9
  riverpod_annotation: ^2.3.3
```

```
# Utilidades de Mapeo JSON
json_annotation: ^4.8.1

dev_dependencies:
  flutter_test:
    sdk: flutter

# Generadores de Código (Herramientas de Desarrollo)
build_runner: ^2.4.8
isar_generator: ^3.1.0+1
riverpod_generator: ^2.3.9
json_serializable: ^6.7.1

flutter:
  uses-material-design: true
```

1. Capa de Dominio (El Contrato)

`lib/domain/models/app_user.dart`

El modelo define nuestra entidad. Para garantizar la integridad, desacoplamos la identidad local de la remota.

```
import 'package:isar/isar.dart';
import 'package:json_annotation/json_annotation.dart';

part 'app_user.g.dart';

@collection
@JsonSerializable()
class AppUser {
  // ID manejado internamente por Isar para evitar colisiones locales
  Id id = Isar.autoIncrement;

  // ID que proviene de la API externa
  @JsonKey(name: 'id')
  int? serverId;

  @JsonKey(name: 'first_name')
  late String name;

  late String email;
  late String avatar;

  AppUser();

  factory AppUser.fromJson(Map<String, dynamic> json) => _$AppUserFromJson(json);
  Map<String, dynamic> toJson() => _$AppUserToJson(this);
```

```
}
```

2. Capa de Configuración (Infraestructura Técnica)

`lib/config/dio_config.dart`

Configuración del cliente HTTP para la comunicación con el servidor externo.

```
import 'package:dio/dio.dart';

class DioConfig {
  static Dio get dio => Dio(
    BaseOptions(
      baseUrl: 'https://reqres.in/api',
      headers: {
        'x-api-key': 'TU_API_KEY_AQUÍ',
        'Content-Type': 'application/json',
      },
    ),
  );
}
```

`lib/config/isar_config.dart`

Configuración de la instancia de la base de datos NoSQL local.

```
import 'package:isar/isar.dart';
import 'package:path_provider/path_provider.dart';
import '../domain/models/app_user.dart';

class IsarConfig {
  static Future<Isar> openDB() async {
    if (Isar.instanceNames.isNotEmpty) return Isar.getInstance();
    final dir = await getApplicationDocumentsDirectory();
    return await Isar.open([AppUserSchema], directory: dir.path);
  }
}
```

3. Capa de Infraestructura (Especialistas y Estrategia)

`lib/infrastructure/datasources/remote_user_datasource.dart`

Su única responsabilidad es realizar la petición de red y mapear el JSON inicial.

```
import '../..../config/dio_config.dart';
import '../..../domain/models/app_user.dart';

class RemoteUserDatasource {
  final _dio = DioConfig.dio;

  Future<List<AppUser>> fetchRemoteUsers() async {
    final response = await _dio.get('/users');
    final List data = response.data['data'];
    return data.map((json) => AppUser.fromJson(json)).toList();
  }
}
```

lib/infrastructure/datasources/isar_datasource.dart

Centraliza la apertura de la base de datos para los repositorios.

```
import '../..../config/isar_config.dart';
import 'package:isar/isar.dart';

class IsarDatasource {
  static Future<Isar> openDB() => IsarConfig.openDB();
}
```

lib/infrastructure/repositories/user_repository_impl.dart

Es el orquestador. Decide de dónde vienen los datos y garantiza que la base de datos local esté siempre actualizada (Single Source of Truth).

```
import '../..../domain/models/app_user.dart';
import '../datasources/isar_datasource.dart';
import '../datasources/remote_user_datasource.dart';
import 'package:isar/isar.dart';

class UserRepositoryImpl {
  final _remote = RemoteUserDatasource();

  Future<List<AppUser>> getAndSyncUsers() async {
    final isar = await IsarDatasource.openDB();
    try {
      final remoteUsers = await _remote.fetchRemoteUsers();

      // Sincronización atómica: limpiar e insertar datos frescos
      await isar.writeTxn(() async {
        await isar.appUsers.clear();
        await isar.appUsers.putAll(remoteUsers);
      });
    }
  }
}
```

```

    });
    return remoteUsers;
  } catch (e) {
    // Si la red falla, servimos la persistencia local
    return await isar.appUsers.where().findAll();
  }
}

Future<void> saveLocal(AppUser user) async {
  final isar = await IsarDatasource.openDB();
  await isar.writeTxn(() => isar.appUsers.put(user));
}
}

```

4. Capa de Presentación (Estado y UI)

`lib/presentation/providers/user_provider.dart`

Utilizamos Riverpod para exponer los datos del repositorio a la interfaz de usuario.

```

import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../domain/models/app_user.dart';
import '../infrastructure/repositories/user_repository_impl.dart';

part 'user_provider.g.dart';

@riverpod
class UserNotifier extends _$UserNotifier {
  final _repo = UserRepositoryImpl();

  @override
  Future<List<AppUser>> build() async => await _repo.getAndSyncUsers();

  Future<void> createUser(String name, String email) async {
    final newUser = AppUser()
      ..name = name
      ..email = email
      ..avatar = 'https://reqres.in/img/faces/7-image.jpg';

    await _repo.saveLocal(newUser);
    ref.invalidateSelf();
  }
}

```

`lib/presentation/screens/user_list_screen.dart`

Construcción de la vista reactiva que maneja estados de carga, éxito y error.

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/user_provider.dart';

class UserListScreen extends ConsumerWidget {
  const UserListScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final usersAsync = ref.watch(userNotifierProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Cyb-Shark: Ingeniería de Datos')),
      body: usersAsync.when(
        data: (users) => ListView.builder(
          itemCount: users.length,
          itemBuilder: (context, i) => ListTile(
            leading: CircleAvatar(backgroundImage: NetworkImage(users[i].avatar)),
            title: Text(users[i].name),
            subtitle: Text(users[i].email),
          ),
        ),
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('Modo Offline: Datos locales
cargados.')),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => ref.read(userNotifierProvider.notifier).createUser('Nuevo
Shark', 'admin@cyb.com'),
        child: const Icon(Icons.person_add),
      ),
    );
  }
}
```

5. Inicialización de la Aplicación

lib/main.dart

Punto de entrada que configura el entorno global.

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'presentation/screens/user_list_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: MyApp()));
}
```

```
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.deepOrange),
      home: const UserListScreen(),
    );
  }
}
```



Conceptos de Ingeniería Aplicados

1. **Desacoplamiento de Identidad:** El uso de un **Id** autoincremental para **Isar** y un **serverId** opcional permite que la base de datos local sea independiente del esquema del servidor.
2. **Datasource vs Repositorio:** El Datasource es un **especialista técnico** (sabe de URLs o motores de DB). El Repositorio es un **estratega de negocio** (sabe qué fuente de datos priorizar según el contexto).
3. **Resiliencia Offline:** El flujo **try-catch** en el repositorio garantiza que el usuario siempre tenga acceso a la información, convirtiendo los fallos de red en estados de "Lectura Local" transparentes.